
CI/CD Pipeline

Continuous Integration & Continuous Deployment
DevOps and Cloud Computing Foundation

Week 6 | Spring 2026 | BS Computer Science

Agenda / Table of Contents

1. Recap: VCS & Containerization
2. Introduction to CI/CD
3. Why CI/CD Matters
4. Continuous Integration (CI)
5. CI Workflow & Best Practices
6. Continuous Delivery vs Deployment
7. CD Workflow & Strategies
8. CI/CD Pipeline Stages
9. CI/CD Tools Landscape
10. GitHub Actions Deep Dive
11. Jenkins Overview
12. Testing in CI/CD
13. Security & Monitoring
14. Interview Prep & Summary

Section 1

Recap: Version Control & Containerization

Version Control System Recap

Git: Distributed version control — tracks every change

Branching & Merging — parallel development workflows

Pull Requests — code review before merging

Remote Repositories — GitHub, GitLab, Bitbucket

Git Flow / Trunk-Based Development — branching strategies



Containerization Recap

Dockerfile — blueprint to build container images

Docker Images & Containers — lightweight, portable runtimes

Docker Volumes — persistent data storage

Docker Networks — container-to-container communication

Docker Compose — multi-container orchestration (YAML)



The Missing Piece: Automation

You know Git (version control) and Docker (containerization)...

But who builds, tests, and deploys your code every time you push?

Answer: A CI/CD Pipeline — the automated bridge.



Section 2

Introduction to CI/CD

What is CI/CD?

CI/CD is a set of practices that automate the process of integrating code changes, running tests, and deploying applications — enabling faster, reliable software delivery.

CI

Continuous Integration
Automate build & test on
every code commit

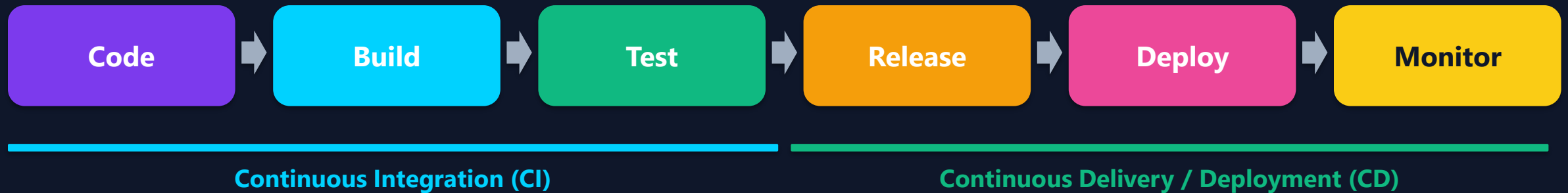
CD

Continuous Delivery
Automate release to staging
Manual approval to prod

CD

Continuous Deployment
Automate release directly
to production (no gate)

CI/CD Pipeline — Big Picture



1. Code — Developer pushes code to the repository (Git)
2. Build — Source code is compiled / Docker image is built
3. Test — Unit, integration, and end-to-end tests run automatically
4. Release — Artifact is versioned and stored (Docker Registry, Nexus)
5. Deploy — Application is deployed to staging / production
6. Monitor — Logs, metrics, and alerts verify the release is healthy

Section 3

Why CI/CD Matters

Without CI/CD — The Pain

"It works on my machine" syndrome

Manual builds — slow, error-prone, inconsistent

Large, risky releases deployed all at once (Big Bang)

Bugs discovered late — expensive to fix

Integration conflicts pile up (merge hell)

Developers afraid to push code frequently

Deployment is a stressful, weekend event

Result: Slow delivery, low quality, unhappy teams

With CI/CD — The Benefits

Faster feedback — know in minutes if code is broken

Smaller, safer releases — deploy many times per day

Automated testing catches bugs early

Consistent environments — Docker + pipeline = reproducible

Less manual work — engineers focus on features

Faster time to market — competitive advantage

Higher confidence in every release

Result: Fast delivery, high quality, happy teams & customers

Before vs After CI/CD

Without CI/CD

- Manual builds on developer machines
- Tests run occasionally (or never)
- Deployment is a manual, multi-hour process
- Releases every few weeks/months
- Bugs found in production
- Blame culture, stressful releases

With CI/CD

- Automated builds on every push
- Tests run automatically, every commit
- One-click (or zero-click) deployments
- Releases multiple times per day
- Bugs caught before they reach users
- Confidence, speed, collaboration

CI/CD by the Numbers

208x

More frequent
deployments

106x

Faster lead time
for changes

7x

Lower change
failure rate

2604x

Faster recovery
from incidents

Source: DORA State of DevOps Report (Google Cloud)

Section 4

Continuous Integration (CI)

What is Continuous Integration?

Continuous Integration (CI) is the practice of frequently merging code changes into a shared repository, where automated builds and tests verify each change.

Key Principles:

- Developers integrate code at least once per day

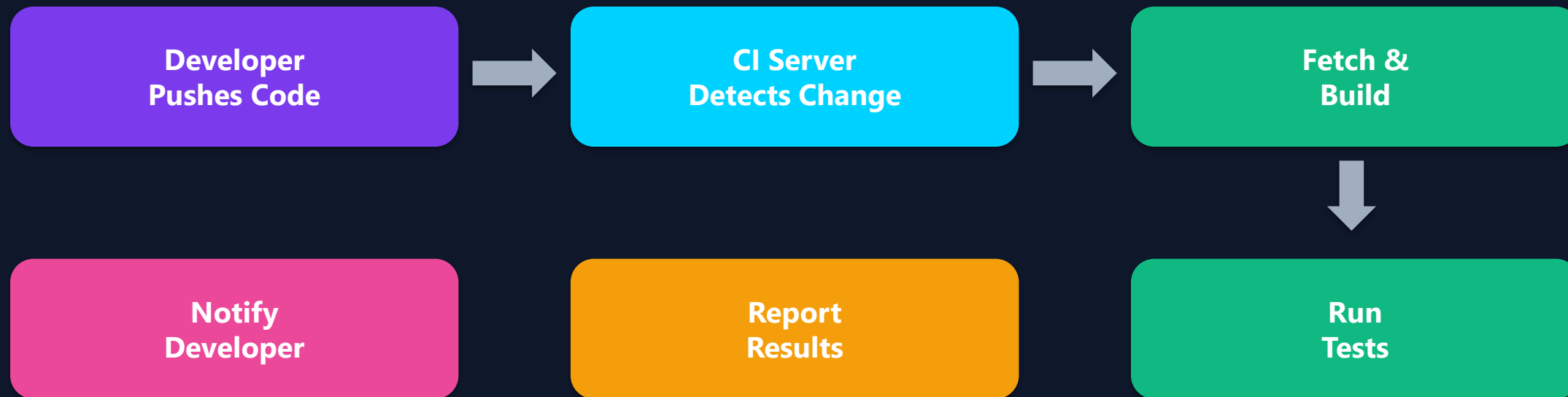
- Every integration triggers an automated build

- Every build runs automated tests

- Broken builds are fixed immediately (top priority)

- The main branch is always in a deployable state

CI Workflow Diagram



Flow: Push → CI detects → Fetch code → Build → Run tests → Report → Notify developer

CI Best Practices

1. Commit Early, Commit Often

Small, frequent commits reduce integration risk

2. Automate the Build

One command should build the entire project

3. Make the Build Self-Testing

Build includes automated test execution

4. Fix Broken Builds Immediately

A broken build is the team's #1 priority

5. Keep the Build Fast

Target < 10 minutes; fast feedback is critical

6. Test in a Clone of Production

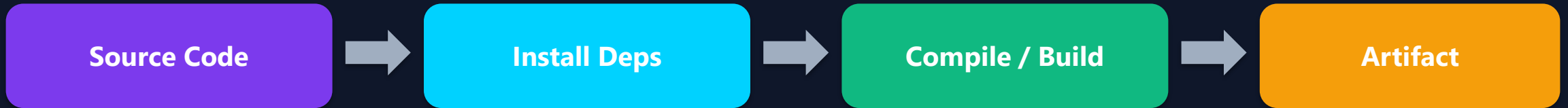
Use Docker to mirror production environment

7. Everyone Can See What's Happening

Build status dashboards, notifications

CI Stage: Build

The Build stage compiles source code, resolves dependencies, and creates deployable artifacts (JAR, Docker image, binary, bundle).



Common Build Tools:

Python: `pip install` → `pytest` / `build`

JavaScript: `npm install` → `npm run build`

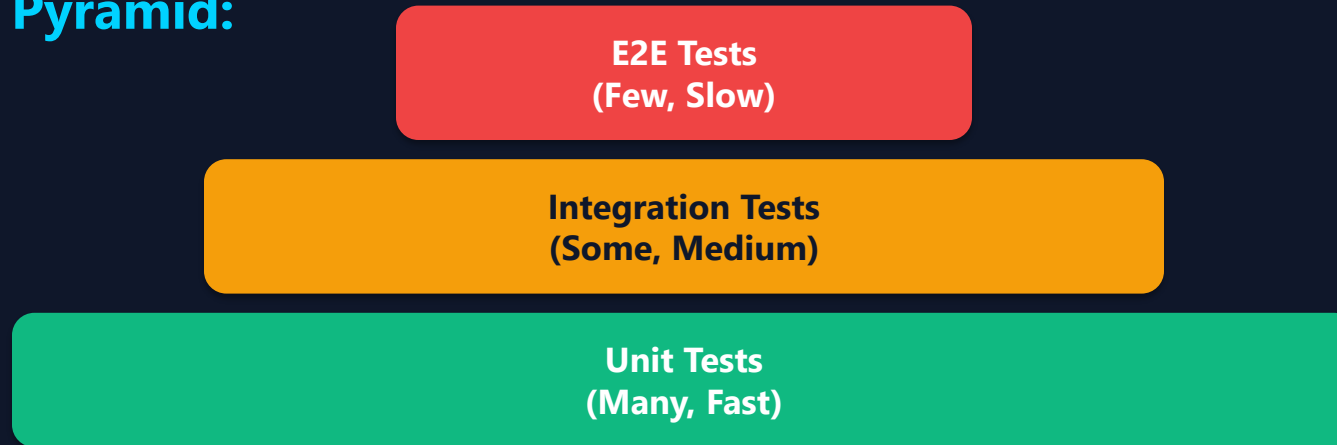
Java: `mvn compile` / `gradle build`

Docker: `docker build -t myapp:latest .`

CI Stage: Test

Automated tests validate that the code works correctly after every change.

Test Pyramid:



Test Types:

Unit Tests

Test individual functions/methods

Integration Tests

Test component interactions

E2E Tests

Test complete user workflows

Smoke Tests

Quick sanity checks

Regression Tests

Ensure old bugs stay fixed

CI Stage: Code Quality & Linting

CI pipelines enforce code quality standards automatically before code is merged.

Linting

Enforce coding standards (ESLint, Pylint, Flake8)

Formatting

Consistent style (Prettier, Black, gofmt)

Static Analysis

Find bugs without running code (SonarQube, CodeQL)

Code Coverage

Measure % of code tested (Istanbul, Coverage.py)

Dependency Audit

Check for known vulnerabilities (npm audit, Snyk)

Section 5

Continuous Delivery & Deployment

Continuous Delivery vs Continuous Deployment

Continuous Delivery

Every change is automatically built, tested, and prepared for release.

Deployment to production requires manual approval (a human clicks the button).

Good for: regulated industries, teams new to CI/CD.



Continuous Deployment

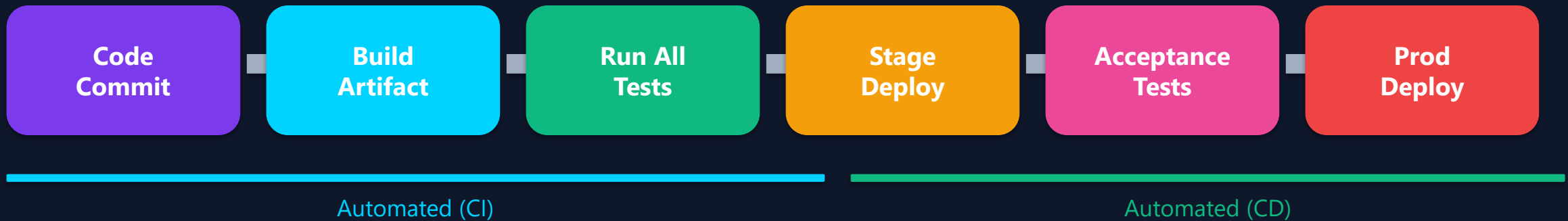
Every change that passes all tests is automatically deployed to production.

No manual gate — fully automated end-to-end.

Good for: mature teams with strong test coverage and monitoring.



CD Workflow Diagram



Key Points:

- Each stage acts as a quality gate — failure stops the pipeline
- Artifacts are built ONCE and promoted through environments
- Staging environment mirrors production (use Docker!)
- Acceptance tests validate business requirements
- Production deployment can be automatic or require approval

Deployment Strategies

Rolling Update

Gradually replace old instances with new ones. Zero downtime. Default in Kubernetes.

Blue/Green

Run two identical environments. Switch traffic from Blue (old) to Green (new) instantly.

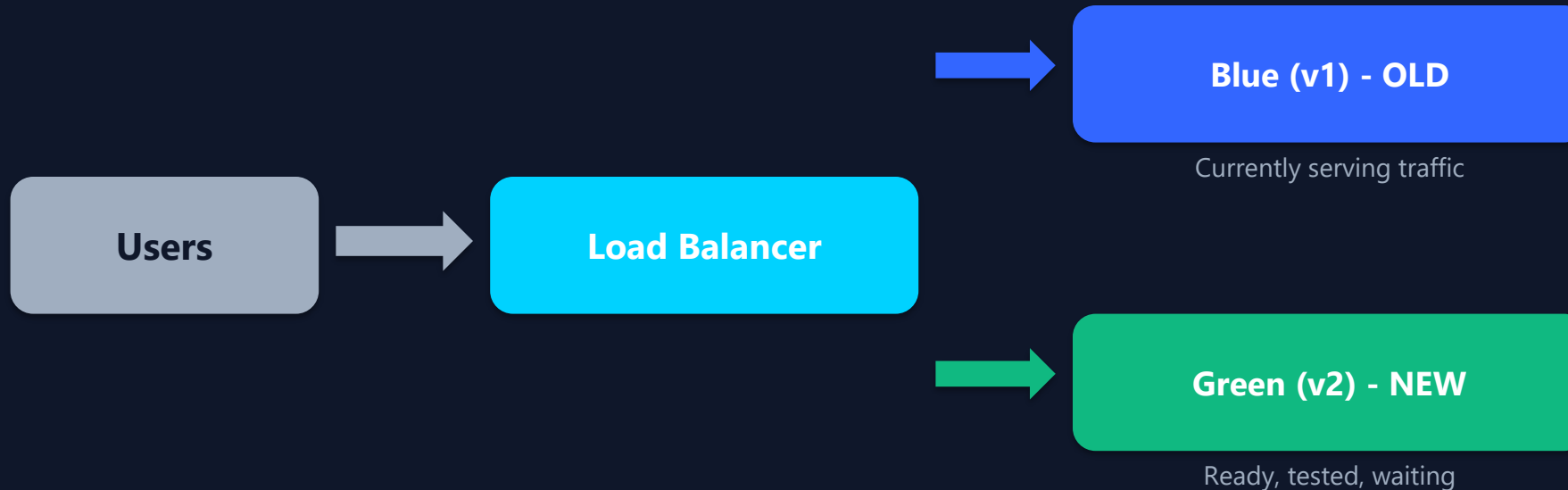
Canary

Deploy to a small % of users first. Monitor, then gradually roll out to 100%.

Recreate

Stop all old instances, deploy new ones. Simple but causes downtime.

Blue/Green Deployment

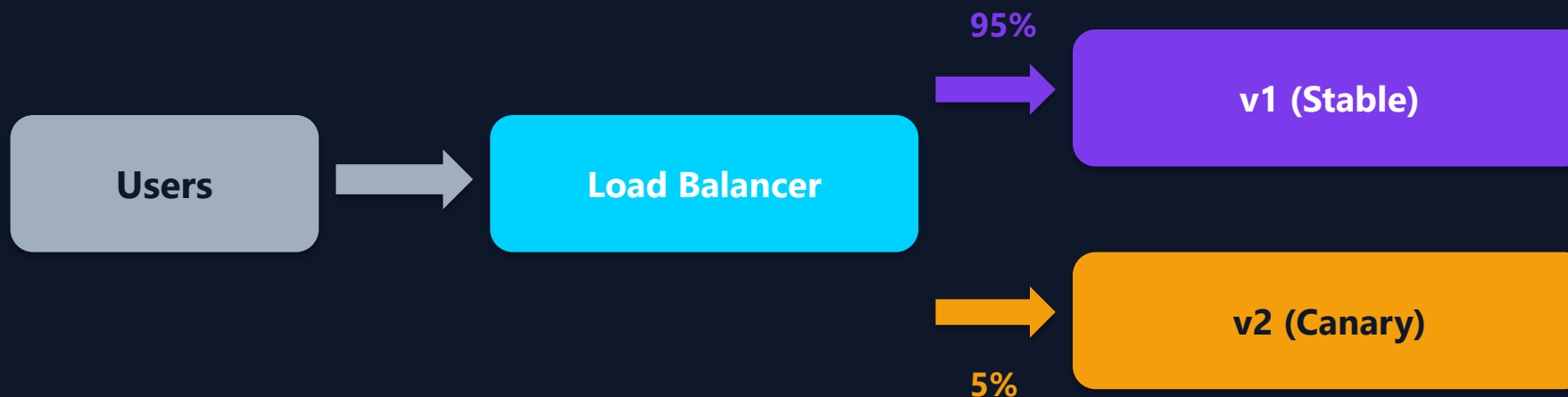


How it works: Deploy v2 to Green → Test Green → Switch Load Balancer to Green

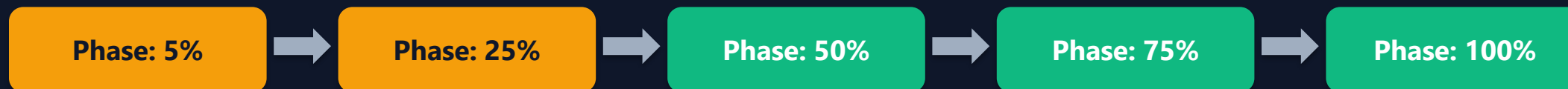
Rollback: Just switch Load Balancer back to Blue (instant rollback!)

Advantage: Zero downtime, instant rollback | Disadvantage: Double infrastructure cost

Canary Deployment



Canary Rollout Phases:



If errors spike at any phase → automatic rollback to v1

Section 6

CI/CD Pipeline Stages in Detail

Anatomy of a CI/CD Pipeline

1. Source

Trigger: git push, PR, schedule | Fetch code from repository

2. Build

Compile code, install dependencies | Create Docker image / artifact

3. Test

Unit tests, integration tests | Code quality, security scans

4. Package

Version the artifact | Push to registry (Docker Hub, ECR)

5. Deploy Staging

Deploy to staging environment | Run acceptance / smoke tests

6. Deploy Prod

Deploy to production | (manual gate or automatic)

7. Monitor

Health checks, logs, metrics | Alert on errors, auto-rollback

Pipeline Triggers

What starts a CI/CD pipeline?

Push to Branch

Pipeline runs on every git push (e.g., push to main or feature/*)

Pull Request

Pipeline runs when a PR is opened or updated — validates before merge

Schedule (Cron)

Pipeline runs on a schedule (e.g., nightly builds, weekly scans)

Manual Trigger

Developer manually starts pipeline (e.g., deploy to production)

Tag / Release

Pipeline runs when a git tag is created (e.g., v1.0.0 release)

Artifacts & Container Registries

An artifact is the output of a build — a deployable unit. Registries store and version these artifacts so they can be retrieved during deployment.



Popular Registries:

Docker Hub — public/private Docker image registry

Amazon ECR — AWS managed container registry

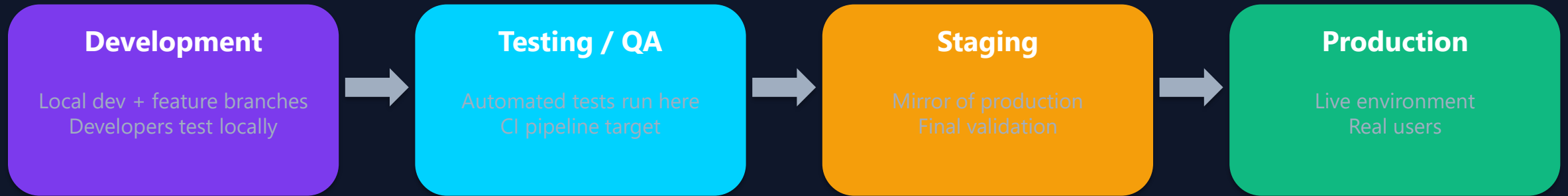
Google Artifact Registry — GCP managed registry

GitHub Container Registry (ghcr.io) — built into GitHub

Azure Container Registry (ACR) — Azure managed registry

Nexus / JFrog Artifactory — universal artifact management

Environments in CI/CD



Best Practices:

- Use Docker to ensure environment parity (same image everywhere)
- Use environment variables for configuration (not hardcoded values)
- Never promote code that hasn't passed tests in lower environments
- Staging should be as close to production as possible

Section 7

CI/CD Tools Landscape

Popular CI/CD Tools

GitHub Actions

Built into GitHub, YAML-based, free for public repos

Jenkins

Open-source, self-hosted, plugin ecosystem, Jenkinsfile

GitLab CI/CD

Built into GitLab, .gitlab-ci.yml, integrated DevOps

CircleCI

Cloud-native, fast builds, Docker-first

Travis CI

Simple YAML config, popular for open-source

Azure DevOps

Microsoft's CI/CD, Pipelines + Boards + Repos

AWS CodePipeline

AWS native, integrates with CodeBuild/CodeDeploy

Argo CD

GitOps-based, Kubernetes-native CD tool

Choosing the Right CI/CD Tool

Key Factors to Consider:

Hosting

Cloud-hosted vs Self-hosted?

GitHub Actions = cloud | Jenkins = self-hosted

Integration

Where is your code?

GitHub → Actions | GitLab → GitLab CI

Cost

Budget and scale?

Open-source projects → GitHub Actions (free)

Complexity

Simple or enterprise?

Small teams → Actions | Enterprise → Jenkins/Azure

Ecosystem

Plugins & marketplace?

Jenkins: 1800+ plugins | Actions: marketplace

Container Support

Docker-native?

All modern tools support Docker natively

Section 8

GitHub Actions — Deep Dive

What is GitHub Actions?

GitHub Actions is a CI/CD platform built directly into GitHub. It allows you to automate workflows — build, test, and deploy — triggered by GitHub events.

Event-driven — triggered by push, PR, schedule, etc.

YAML-based configuration — `.github/workflows/*.yml`

Free for public repositories, generous free tier for private

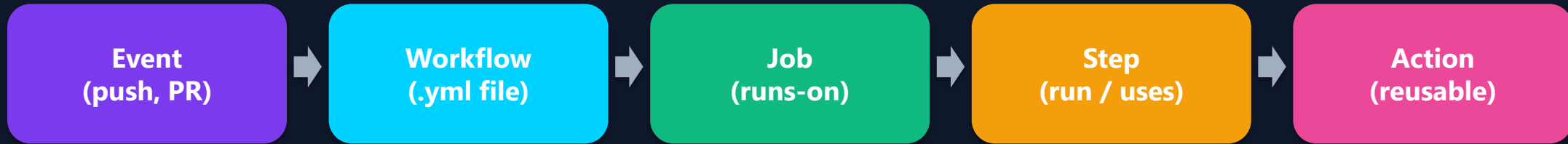
Marketplace with 15,000+ pre-built actions

Supports Linux, macOS, and Windows runners

Native Docker and container support

Matrix builds for testing across multiple versions

GitHub Actions Architecture



Key Concepts:

Event	What triggers the workflow (push, pull_request, schedule, workflow_dispatch)
Workflow	A YAML file in .github/workflows/ — defines the automation
Job	A set of steps that run on the same runner (virtual machine)
Step	Individual task within a job — run a command or use an action
Action	Reusable unit of code (from marketplace or custom)
Runner	The machine (GitHub-hosted or self-hosted) that executes the job

GitHub Actions — Workflow YAML

```
name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm install
      - run: npm test
```

name:

Workflow name (shown in UI)

on:

Event triggers

jobs:

Define one or more jobs

runs-on:

Runner (ubuntu, windows, macos)

steps:

Sequential tasks in the job

uses:

Use a pre-built action

run:

Execute a shell command

GitHub Actions — Docker Build & Push

```
name: Docker Build & Push
on:
  push:
    branches: [ main ]

jobs:
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USER }
          password: ${ secrets.DOCKER_PASS }

      - name: Build and Push
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: myapp:latest
```

Key Highlights:

Uses pre-built Docker actions from marketplace

Secrets stored securely in GitHub Settings → Secrets

`${ secrets.* }` injects values at runtime

Builds Docker image from Dockerfile in repo root

Pushes image to Docker Hub automatically on merge

GitHub Actions — Secrets & Variables

Never hardcode passwords, API keys, or tokens in your workflow files!

Secrets

- Encrypted at rest, masked in logs
- Set in: Settings → Secrets → Actions
- Access: `${{ secrets.MY_SECRET }}`
- Use for: passwords, tokens, keys
- Cannot be viewed after creation
- Repository or Organization level

Variables

- Plain text, visible in logs
- Set in: Settings → Variables → Actions
- Access: `${{ vars.MY_VAR }}`
- Use for: config values, URLs, flags
- Can be viewed and edited anytime
- Repository or Environment level

GitHub Actions — Matrix Builds

Matrix strategy runs your job across multiple configurations in parallel.

```
jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        node-version: [18, 20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node-version }
      - run: npm test
```

This creates 6 parallel jobs:

Ubuntu + Node 18

Ubuntu + Node 20

Ubuntu + Node 22

Windows + Node 18

Windows + Node 20

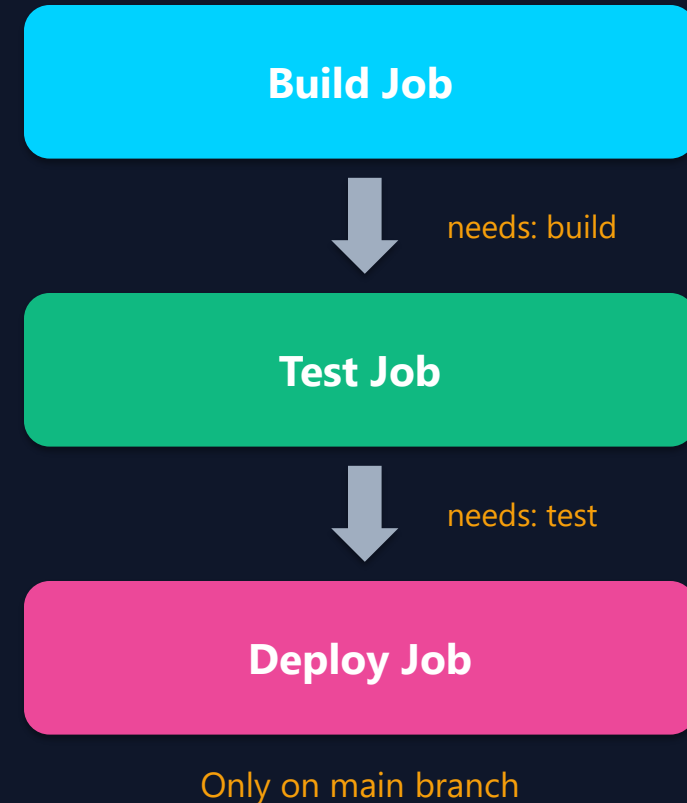
Windows + Node 22

GitHub Actions — Multi-Job Pipeline

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install && npm run build

  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - run: echo 'Deploying...'
```



Section 10

Testing in CI/CD Pipelines

Why Automated Testing is Critical

Without automated tests, CI/CD is just Continuous Integration of bugs. Tests are the safety net that makes fast, frequent deployments possible.

Catch Bugs Early

Bugs found in CI cost 10x less than bugs in production

Enable Refactoring

Tests give confidence to change code without breaking it

Documentation

Tests describe expected behavior — living documentation

Faster Feedback

Developers know within minutes if their change is safe

Deploy Confidence

100% automated tests pass → safe to deploy

Types of Tests in CI/CD

Unit Tests

Test individual functions in isolation
Fastest, most numerous
Tools: Jest, pytest, JUnit

Integration Tests

Test how components work together
Database, API, service interactions
Tools: Supertest, TestContainers

E2E Tests

Test complete user workflows
Slowest, fewest, most realistic
Tools: Cypress, Playwright, Selenium

Security Tests

SAST, DAST, dependency scanning
Find vulnerabilities automatically
Tools: Snyk, OWASP ZAP, CodeQL

Test Execution Order in Pipeline: Unit → Integration → E2E → Security

Principle: Run fastest tests first — fail fast, save time and resources.

Code Coverage in CI/CD

Code coverage measures what percentage of your code is executed during tests.

Line Coverage

% of code lines executed

85%

Branch Coverage

% of if/else branches tested

75%

Function Coverage

% of functions called

90%

Statement Coverage

% of statements executed

82%

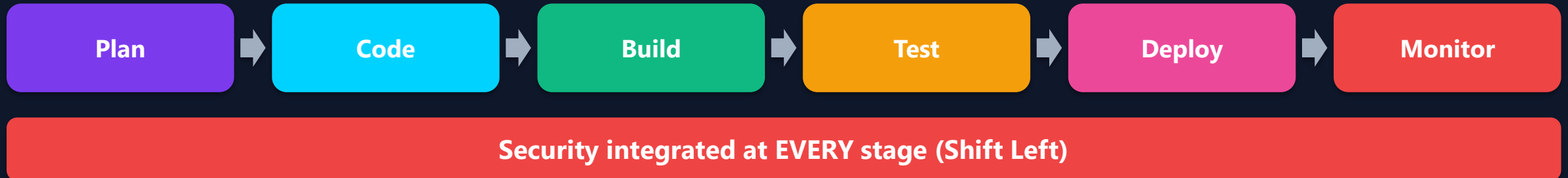
Tip: Aim for 70-80% coverage. 100% is impractical — focus on critical paths.

Section 11

Security in CI/CD (DevSecOps)

DevSecOps — Security Shifted Left

DevSecOps integrates security into every phase of the CI/CD pipeline, rather than treating it as an afterthought. This is called 'shifting left'.



SAST	Static Application Security Testing — scan source code
DAST	Dynamic Testing — test running application
SCA	Software Composition Analysis — check dependencies
Container Scan	Scan Docker images for vulnerabilities (Trivy, Snyk)
Secret Detection	Prevent secrets from being committed (GitLeaks)

Security Checks in a Pipeline

```
name: Security Pipeline
on: [push, pull_request]

jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run SAST scan
        uses: github/codeql-action/analyze@v3

      - name: Dependency audit
        run: npm audit --audit-level=high

      - name: Scan Docker image
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'myapp:latest'
          severity: 'HIGH,CRITICAL'
```

Security Gates:

CodeQL

GitHub's SAST engine
Finds SQL injection, XSS, etc.

npm audit

Checks dependencies for
known CVEs

Trivy

Scans Docker images for
OS & library vulnerabilities

GitLeaks

Detects hardcoded secrets
in source code

Section 12

Monitoring, Rollback & Observability

Post-Deployment Monitoring

Deployment is NOT the finish line. Monitoring ensures your release is healthy.

Metrics

CPU, memory, response time,
error rate, request count
Tools: Prometheus, Grafana

Logs

Application logs, structured logging, centralized collection
Tools: ELK Stack, Loki

Traces

Request flow across services, latency breakdown, bottlenecks
Tools: Jaeger, Zipkin

Alerts

Notify team when thresholds are breached (error rate > 1%)
Tools: PagerDuty, Opsgenie

The Three Pillars of Observability: Metrics + Logs + Traces
Together they answer: What happened? Why? Where?

Rollback Strategies

When a deployment goes wrong, you need to revert quickly and safely.

Revert Commit

Git revert → push → pipeline redeploys previous version
Simplest approach, works with any CI/CD tool

Blue/Green Switch

Switch load balancer back to Blue environment
Instant rollback, zero downtime

Container Rollback

Redeploy previous Docker image tag (e.g., v1.2.3)
Docker makes this trivial — images are immutable

Feature Flags

Toggle off the broken feature without redeploying
No rollback needed — just flip the flag

Auto-Rollback

Monitoring detects errors → automatically triggers rollback
Most mature approach (requires good monitoring)

Section 13

Real-World CI/CD Examples

Case Study: How Netflix Does CI/CD

Netflix deploys thousands of times per day to serve 230M+ subscribers.

Spinnaker

Open-source CD platform (created by Netflix)

Canary Analysis

Every deployment is tested on a small % of traffic first

Automated Rollback

If canary metrics degrade → automatic rollback

Immutable Infra

Never update servers — always deploy fresh instances

Chaos Engineering

Netflix Chaos Monkey randomly kills services to test resilience

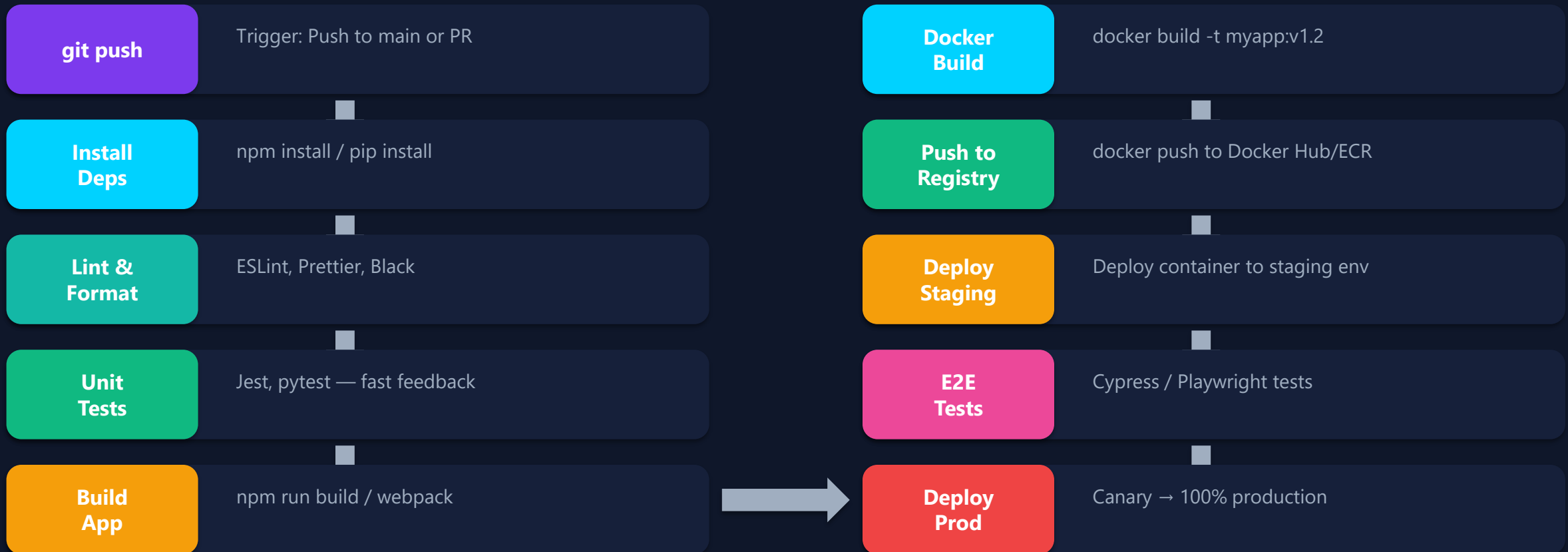
Microservices

Hundreds of independent services, each with its own pipeline

Key Takeaway: CI/CD at scale requires strong testing, monitoring, and automation.

Real-World Pipeline: Full Stack App

A typical pipeline for a React + Node.js + Docker application:



GitLab CI/CD — Quick Overview

```
# .gitlab-ci.yml
stages:
  - build
  - test
  - deploy

build:
  stage: build
  image: node:20
  script:
    - npm install
    - npm run build

test:
  stage: test
  script:
    - npm test

deploy:
  stage: deploy
  script:
    - ./deploy.sh
  only:
    - main
```

GitLab CI Key Features:

Built into GitLab (no extra setup)

Single .gitlab-ci.yml config file

Docker-native — each job runs in a container

Auto DevOps — zero-config pipelines

Built-in container registry

Environment management UI

Built-in security scanning (SAST, DAST)

Free CI/CD minutes for all plans

Section 14

Pipeline as Code & Best Practices

Pipeline as Code

Pipeline as Code means defining your CI/CD pipeline in a version-controlled file that lives alongside your application code — not configured through a UI.

Version Controlled	Pipeline changes are tracked in Git — full audit trail
Code Review	Pipeline changes go through the same PR process as code
Reproducible	Same pipeline runs everywhere — no snowflake servers
Portable	Move between environments without manual reconfiguration
Self-Documenting	New team members read the pipeline file to understand the flow
Testable	Pipeline changes can be tested in branches before merging

GitHub: `.github/workflows/*.yml` | **Jenkins:** `Jenkinsfile` | **GitLab:** `.gitlab-ci.yml`

CI/CD Best Practices

Keep pipelines fast

< 10 min feedback loop — cache dependencies, parallelize

Fail fast

Run cheapest/fastest checks first (lint → unit → integration)

Build once, deploy many

Same artifact flows through all environments

Use Docker everywhere

Consistent environments from dev to production

Secure your pipeline

Scan for vulnerabilities, manage secrets properly

Monitor after deploy

Don't just deploy and pray — watch metrics & logs

Automate everything

If you do it twice, automate it the third time

Small, frequent changes

Small PRs are easier to review, test, and rollback

CI/CD Anti-Patterns to Avoid

Slow Pipelines

Pipelines > 30 min — developers stop waiting, ignore failures

Flaky Tests

Tests that randomly pass/fail — erode trust in CI

Manual Steps

Any manual step is a bottleneck and error source

No Environment Parity

"Works in staging but not prod" — use Docker!

Secret in Code

Hardcoded credentials — use secret management

Skipping Tests

"I'll add tests later" — later never comes

Ignoring Failures

Red builds become normal — broken window syndrome

Thank You!

Questions & Discussion

Next Steps:

1. Set up a GitHub Actions workflow for your project
2. Add automated tests to your pipeline
3. Build and push a Docker image via CI/CD
4. Practice with the hands-on lab exercises